SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – CODING CHALLENGE

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 3 – Coding Challenge
Weightage:	30% of CIA		
CO5 Statement:	Analyse NLP models, regular expression patterns, finite state automata, morphological processing techniques and to interpret language processing. (BL-3)		
Student Name:	B.Archana	**Reg. No.:**	192425325
Section / Batch:	2024	**Date:**	21-10-2026

Code of Conduct

I __B.Archana(192425325)_____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: ____B.Archana_____

Instructions

- Develop a complete and modular Python program that meets all the functional requirements specified in the coding challenge using appropriate algorithms, data structures, and programming practices.
- Test the program using multiple valid and invalid test cases, and clearly display the input, processing, and output to verify the correctness of the implementation.
- Follow standard coding practices by using meaningful variable names, proper indentation, comments, exception handling (where applicable), and upload the source code, sample outputs, and README file to the designated GitHub repository.
- Duration: 40 minutes.

Section / Topic	Focus	Q Nos
Regular Expression Pattern Matching	Regular Expressions, Basic Regular Expression Patterns	1
Finite State Automata Implementation	Finite State Automata, Models and Algorithms	2
Advanced Regular Expression Search	Regular Expressions, Basic Regular Expression Patterns	3

Annexure A – Coding Challenge

Section 1: Intelligent Email and Password Validator using Regular Expressions

1. **Problem Statement:**

A software company is developing a user registration system. Before creating an account, the system must validate user credentials based on predefined security policies.

Develop a Python application that validates:
Email Address
Password
Mobile Number

using Regular Expressions.

Validation Rules
Email
Must start with an alphabet.
May contain letters, digits, '.', '_' before '@'.
Domain should contain only alphabets.
Valid extensions: .com, .org, .edu, .net, .in
Password
Minimum 8 characters
At least one uppercase letter
At least one lowercase letter
At least one digit
At least one special character (@,#,$,%,&,!)
Mobile Number
Must contain exactly 10 digits.
First digit should be between 6–9.
Expected Output

Display:

Valid Email / Invalid Email
Strong Password / Weak Password
Valid Mobile Number / Invalid Mobile Number

Section 2: Design a Finite State Automata (FSA) Simulator

2. **Problem Statement**

Develop a Python-based Deterministic Finite Automata (DFA) simulator.

The program should:

- Accept the DFA description
    - States
    - Input alphabet
    - Transition table
    - Initial state
    - Final state(s)
- Accept multiple input strings.
- Simulate state transitions.
- Display whether the string is Accepted or Rejected.

Example

Language:
Strings ending with "ab"

Input
abaab

Output
Transition Path:
q0 → q1 → q0 → q1 → q2
Accepted

Section 3: Smart Pattern Matching Engine using Regular Expressions

3. Problem Statement

Develop a Python-based text search engine that performs pattern matching using Regular Expressions.

The system should support:
Word Search
Prefix Search
Suffix Search
Date Search
Phone Number Search
Hashtag Search
Mention (@username) Search

Example Input
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing

User Menu
1.Search Date
2.Search Phone Number
3.Search Hashtag
4.Search Mention
5.Search Prefix
6.Search Suffix

Expected Output
Display all matching patterns based on user selection.

Question 1: User Registration Validation Using Regular Expressions in Python

Aim:

To develop a Python application using Regular Expressions to validate an email address, password, and mobile number based on predefined security rules.

Algorithm:

1. Start the program.
2. Import the re module.
3. Define a Regular Expression pattern for email validation.
4. Define a Regular Expression pattern for password validation.
5. Define a Regular Expression pattern for mobile number validation.
6. Get the email address, password, and mobile number from the user.
7. Validate the email address and display Valid Email or Invalid Email.
8. Validate the password and display Strong Password or Weak Password.
9. Validate the mobile number and display Valid Mobile Number or Invalid Mobile Number.
10. Stop the program.

Python Program:

```
import re
# Regular expression patterns
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!])[A-Za-z\d@#$%&!]{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
# Get input from the user
email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
# Email validation
if re.fullmatch(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
# Password validation
if re.fullmatch(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
# Mobile number validation
```

```
if re.fullmatch(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output:

```
= RESTART: C:\Users\ARCHANA\OneDrive\Pictures\Documents\Desktop\NLP\question -1 -AT-3.py
Enter Email Address: archana_b23@college.edu
Enter Password: Archana@456
Enter Mobile Number: 8765432109
Valid Email
Strong Password
Valid Mobile Number
```

Explanation:

- **Email validation:** The email must start with an alphabet and end with one of the permitted extensions: .com, .org, .edu, .net, or .in.
- **Password validation:** The password must contain at least eight characters, including an uppercase letter, a lowercase letter, a digit, and one allowed special character (@, #, $, %, &, !).
- **Mobile number validation:** The mobile number must contain exactly 10 digits and start with a digit between 6 and 9.
- **Regular Expressions:** The Python re module checks whether the entered values match the defined patterns.

Result:

Thus, the Python application was successfully developed and executed to validate email addresses, passwords, and mobile numbers using Regular Expressions according to the given validation rules.

Question 2 : DFA Simulator Using Python

Aim:

To develop a Python program that accepts a DFA description, including states, input alphabet, transition table, initial state, and final states, and simulates multiple input strings to determine whether each string is Accepted or Rejected.

Algorithm:

1. Start the program.
2. Read the set of states and input alphabet.
3. Read the transition table for each state and input symbol.
4. Read the initial state and final state(s).
5. Accept multiple input strings from the user.
6. Set the current state to the initial state for each string.
7. Read each input symbol and follow the corresponding transition.
8. Record and display the transition path.
9. If the final state reached belongs to the set of final states, display Accepted; otherwise, display Rejected.
10. Repeat the process for all input strings.
11. Stop the program.

Python Program:

```
# DFA Simulator in Python
states = input("Enter states (space-separated): ").split()
alphabet = input("Enter input alphabet (space-separated): ").split()
transition = {}
print("Enter transition table:")
for state in states:
    transition[state] = {}
    for symbol in alphabet:
        next_state = input(
            f"Transition from {state} on {symbol}: "
        )
        transition[state][symbol] = next_state
initial_state = input("Enter initial state: ")
final_states = input(
    "Enter final states (space-separated): "
).split()

n = int(input("Enter number of input strings: "))
for i in range(n):
```

```
    string = input(f"Enter string {i + 1}: ")
    current_state = initial_state
    path = [current_state]
    valid = True
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        current_state = transition[current_state][symbol]
        path.append(current_state)
    print("Transition Path:", " -> ".join(path))
    if valid and current_state in final_states:
        print("Accepted")
    else:
        print("Rejected")
```

Output:

```
= RESTART: C:/Users/ARCHANA/OneDrive/Pictures/Documents,
Enter states (space-separated): q0 q1 q2
Enter input alphabet (space-separated): a b
Enter transition table:
Transition from q0 on a: q1
Transition from q0 on b: q0
Transition from q1 on a: q1
Transition from q1 on b: q2
Transition from q2 on a: q1
Transition from q2 on b: q0
Enter initial state: q0
Enter final states (space-separated): q2
Enter number of input strings: 3
Enter string 1: abaab
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2
Accepted
Enter string 2: aabb
Transition Path: q0 -> q1 -> q1 -> q2 -> q0
Rejected
Enter string 3: abab
Transition Path: q0 -> q1 -> q2 -> q1 -> q2
Accepted
|
```

Result:

Thus, the Python-based DFA simulator was successfully developed to accept a DFA description, simulate state transitions for multiple input strings, display the transition path, and determine whether each string is Accepted or Rejected.

Question 3: Text Search Engine Using Regular Expressions in Python

Aim:

To develop a Python-based text search engine using Regular Expressions to search for words, prefixes, suffixes, dates, phone numbers, hashtags, and mentions from a given text based on user selection.

Algorithm:

1. Start the program.
2. Import the re module.
3. Accept the text to be searched from the user.
4. Display a menu containing the search options.
5. Read the user's choice.
6. Define the appropriate Regular Expression pattern for the selected search option.
7. Search the text using the re.findall() function.
8. Display all matching patterns found in the text.
9. If no match is found, display an appropriate message.
10. Repeat the process until the user chooses to exit.
11. Stop the program.

Python Program:

```
import re
text = """
Meeting on 12/09/2026
Call 9876543210
#NLP
@OpenAI
natural language processing
"""
while True:
    print("\n1. Search Date")
    print("2. Search Phone Number")
    print("3. Search Hashtag")
    print("4. Search Mention")
    print("5. Search Prefix")
    print("6. Search Suffix")
    print("7. Search Word")
    print("8. Exit")
    choice = input("Enter choice: "
    if choice == "1":
        pattern = r"\b\d{2}/\d{2}/\d{4}\b"
```

```
elif choice == "2":
    pattern = r"\b[6-9]\d{9}\b"
elif choice == "3":
    pattern = r"(?<!\w)#\w+"
elif choice == "4":
    pattern = r"(?<!\w)@\w+"
elif choice == "5":
    prefix = input("Enter prefix: ")
    pattern = r"\b" + re.escape(prefix) + r"\w*"
elif choice == "6":
    suffix = input("Enter suffix: ")
    pattern = r"\b\w*" + re.escape(suffix) + r"\b"
elif choice == "7":
    word = input("Enter word: ")
    pattern = r"\b" + re.escape(word) + r"\b"
elif choice == "8":
    print("Program ended.")
    break
else:
    print("Invalid choice!")
    continue
matches = re.findall(pattern, text, re.IGNORECASE)
if matches:
    print("Matching Results:")
    for item in matches:
        print(item)
    print("Total Matches:", len(matches))
else:
    print("No matches found.")
```

Output:

```
1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 1
Matching Results:
12/09/2026
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 2
Matching Results:
9876543210
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 3
Matching Results:
#NLP
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 4
Matching Results:
@OpenAI
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 5
Enter prefix: lang
Matching Results:
language
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 6
Enter suffix: processing
Matching Results:
processing
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 7
Enter word: natural
Matching Results:
natural
Total Matches: 1

1. Search Date
2. Search Phone Number
3. Search Hashtag
4. Search Mention
5. Search Prefix
6. Search Suffix
7. Search Word
8. Exit
Enter choice: 8
Program ended.
```

Explanation:

- Word Search: Finds an exact word in the text.
- Prefix Search: Finds words beginning with the specified prefix.
- Suffix Search: Finds words ending with the specified suffix.
- Date Search: Finds dates in DD/MM/YYYY format.
- Phone Number Search: Finds 10-digit mobile numbers beginning with 6, 7, 8, or 9.
- Hashtag Search: Finds hashtags such as #NLP.
- Mention Search: Finds usernames such as @OpenAI.
- Regular Expressions: The re module identifies patterns, and re.findall() returns all matches.

Result:

Thus, the Python-based text search engine was successfully developed using Regular Expressions to search and display matching dates, phone numbers, hashtags, mentions, prefixes, suffixes, and words according to the user's selection.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Q. No. / Criteria	Problem Understanding	Algorithm	Code Implementation	Competitive Performance	Test Case Handling	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____